

Sistema de Gestión de Stock

Implementación ddd

Campo	Detalle
Proyecto	ERP Básico – SDI- Sistema Distribuido de Inventario
Documento	Implementación simple y robusta con auditoría simple
Versión	1.0
Fecha	13-07-2026
Autor	Alberto Fernández

1. Alcance de la primera implementación

Esta implementación prioriza un sistema funcional, estable y fácil de desarrollar. Incluye administración de productos, stock actual, reservas de stock, confirmación de ventas, procesamiento de compras desde cola, movimientos, precios, validaciones, transacciones, control de concurrencia y auditoría simple. No incluye seguridad avanzada, autenticación de usuarios ni control de roles.

- Comunicación síncrona con Ventas para operaciones de consulta, reserva y confirmación.
- Comunicación asíncrona con Compras mediante una cola de mensajes y un consumer/listener.
- Persistencia en PostgreSQL, aprovechando transacciones, claves foráneas, constraints e índices.
- Implementación de logs técnicos para errores, operaciones críticas y reprocesos.

2. Stack tecnológico recomendado

Componente	Tecnología	Uso
Backend/API	Python + FastAPI	Servicios REST, validaciones y documentación automática OpenAPI/Swagger.
Base de datos	PostgreSQL	Transacciones, integridad referencial, concurrencia y consultas.
ORM	SQLAlchemy	Mapeo de tablas y consultas desde Python.
Migraciones	Alembic	Control de cambios de estructura de base de datos.
Mensajería	RabbitMQ	Cola de compras para procesamiento asíncrono.
Servidor aplicación	Uvicorn/Gunicorn	Ejecución del servicio FastAPI.
Proxy	Nginx	Publicación del servicio y manejo de tráfico HTTP.
Pruebas	Pytest	Pruebas unitarias y de integración.
Logs	logging de Python	Registro de eventos y errores.

3. Modelo funcional resumido

Proceso	Descripción	Resultado esperado
Productos	Crear, consultar y listar productos activos.	Catálogo disponible para compras, ventas y stock.
Compra	El consumer lee la cola de compras e incrementa stock.	Entrada registrada y stock actualizado.
Reserva	Ventas solicita comprometer temporalmente cantidades.	Cantidad reservada y no disponible para otras ventas.
Confirmación de venta	La reserva se convierte en salida definitiva.	Stock total disminuido y movimiento de venta registrado.
Liberación de reserva	Si la venta no se concreta, se devuelve al disponible.	Reserva liberada y stock disponible actualizado.
Consultas	Consultar productos, stock y movimientos.	Información consistente para otros módulos.

4. Estructura de base de datos

La estructura propuesta usa tablas simples, con claves primarias, claves foráneas e índices orientados a las consultas principales. La tabla STOCK_ACTUAL concentra el saldo operativo; MOVIMIENTOS_STOCK conserva trazabilidad; RESERVAS_STOCK controla stock comprometido por ventas pendientes.

Tabla	Responsabilidad	Clave primaria
PRODUCTOS	Catálogo maestro de productos.	producto_id
STOCK_ACTUAL	Saldo total, reservado y disponible por producto.	producto_id
RESERVAS_STOCK	Reservas asociadas a documentos de venta.	reserva_id
MOVIMIENTOS_STOCK	Historial de entradas, salidas y cambios.	movimiento_id
PRECIOS_COMPRA	Historial de precios de compra.	precio_compra_id
PRECIOS_VENTA	Historial de precios de venta.	precio_venta_id
COLA_COMPRAS_LOG	Control de mensajes procesados desde la cola.	evento_id

SQL real listo para ejecutar

```
-- =====
-- STK-ERP - Estructura inicial PostgreSQL
-- =====

CREATE TABLE productos (
    producto_id BIGSERIAL PRIMARY KEY,
    codigo VARCHAR(50) NOT NULL UNIQUE,
    nombre VARCHAR(150) NOT NULL,
    descripcion TEXT,
    categoria VARCHAR(100),
    unidad_medida VARCHAR(30) NOT NULL DEFAULT 'UNIDAD',
    activo BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    created_by VARCHAR(80),
    updated_by VARCHAR(80),
    CONSTRAINT chk_producto_codigo_no_vacio CHECK (trim(codigo) <> ''),
    CONSTRAINT chk_producto_nombre_no_vacio CHECK (trim(nombre) <> '')
);

CREATE TABLE stock_actual (
    producto_id BIGINT PRIMARY KEY REFERENCES productos(producto_id),
    cantidad_total NUMERIC(14,3) NOT NULL DEFAULT 0,
    cantidad_reservada NUMERIC(14,3) NOT NULL DEFAULT 0,
    cantidad_disponible NUMERIC(14,3) GENERATED ALWAYS AS (cantidad_total - cantidad_reservada) STORED,
    fecha_ultima_actualizacion TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_by VARCHAR(80),
    CONSTRAINT chk_stock_total_no_negativo CHECK (cantidad_total >= 0),
    CONSTRAINT chk_stock_reservado_no_negativo CHECK (cantidad_reservada >= 0),
    CONSTRAINT chk_stock_reserva_no_mayor_total CHECK (cantidad_reservada <= cantidad_total)
);

CREATE TABLE reservas_stock (
    reserva_id BIGSERIAL PRIMARY KEY,
    producto_id BIGINT NOT NULL REFERENCES productos(producto_id),
    documento_ref VARCHAR(80) NOT NULL,
    cantidad_reservada NUMERIC(14,3) NOT NULL,
    estado_reserva VARCHAR(20) NOT NULL DEFAULT 'PENDIENTE',
    fecha_reserva TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    fecha_confirmacion TIMESTAMP,
    fecha_liberacion TIMESTAMP,
    motivo_liberacion VARCHAR(200),
    created_by VARCHAR(80),
    updated_by VARCHAR(80),
    CONSTRAINT chk_reserva_cantidad_positiva CHECK (cantidad_reservada > 0),
    CONSTRAINT chk_reserva_estado CHECK (estado_reserva IN ('PENDIENTE','CONFIRMADA','LIBERADA')),
);
```

```

CONSTRAINT chk_reserva_doc_no_vacio CHECK (trim(documento_ref) <> '')
);

CREATE TABLE movimientos_stock (
    movimiento_id BIGSERIAL PRIMARY KEY,
    producto_id BIGINT NOT NULL REFERENCES productos(producto_id),
    reserva_id BIGINT REFERENCES reservas_stock(reserva_id),
    tipo_movimiento VARCHAR(30) NOT NULL,
    origen VARCHAR(30) NOT NULL,
    documento_ref VARCHAR(80) NOT NULL,
    cantidad NUMERIC(14,3) NOT NULL,
    stock_anterior NUMERIC(14,3) NOT NULL,
    stock_posterior NUMERIC(14,3) NOT NULL,
    fecha_movimiento TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    observacion TEXT,
    created_by VARCHAR(80),
    CONSTRAINT chk_mov_cantidad_positiva CHECK (cantidad > 0),
    CONSTRAINT chk_mov_tipo CHECK (tipo_movimiento IN
('COMPRA','RESERVA','VENTA','LIBERACION_RESERVA','AJUSTE')),
    CONSTRAINT chk_mov_origen CHECK (origen IN ('COMPRAS','VENTAS','STOCK','SISTEMA')),
    CONSTRAINT chk_mov_doc_no_vacio CHECK (trim(documento_ref) <> '')
);

CREATE TABLE precios_compra (
    precio_compra_id BIGSERIAL PRIMARY KEY,
    producto_id BIGINT NOT NULL REFERENCES productos(producto_id),
    fechaPrecio TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    precio_compra NUMERIC(14,2) NOT NULL,
    documento_ref VARCHAR(80) NOT NULL,
    proveedor_ref VARCHAR(80),
    CONSTRAINT chkPrecio_compra_positivo CHECK (precio_compra >= 0)
);

CREATE TABLE precios_venta (
    precio_venta_id BIGSERIAL PRIMARY KEY,
    producto_id BIGINT NOT NULL REFERENCES productos(producto_id),
    fechaPrecio TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    precio_venta NUMERIC(14,2) NOT NULL,
    documento_ref VARCHAR(80) NOT NULL,
    CONSTRAINT chkPrecio_venta_positivo CHECK (precio_venta >= 0)
);

CREATE TABLE cola_compras_log (
    evento_id BIGSERIAL PRIMARY KEY,
    mensaje_id VARCHAR(100) NOT NULL UNIQUE,
    producto_id BIGINT REFERENCES productos(producto_id),
    referencia_compra VARCHAR(80) NOT NULL,
    cantidad NUMERIC(14,3) NOT NULL,
    precio_compra NUMERIC(14,2) NOT NULL,
    fecha_evento TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    estado_procesamiento VARCHAR(20) NOT NULL DEFAULT 'PENDIENTE',
    error_detalle TEXT,
    intentos INTEGER NOT NULL DEFAULT 0,
    fecha_procesamiento TIMESTAMP,
    CONSTRAINT chkCola_estado CHECK (estado_procesamiento IN ('PENDIENTE','PROCESADO','ERROR')),
    CONSTRAINT chkCola_cantidad_positiva CHECK (cantidad > 0),
    CONSTRAINT chkCola_precio_no_negativo CHECK (precio_compra >= 0)
);

-- Índices recomendados
CREATE INDEX idx_productos_nombre ON productos(nombre);
CREATE INDEX idx_stock_disponible ON stock_actual(producto_id, cantidad_disponible);
CREATE INDEX idx_reservas_producto_estado ON reservas_stock(producto_id, estado_reserva);
CREATE INDEX idx_reservas_documento ON reservas_stock(documento_ref);
CREATE INDEX idx_reservas_fecha ON reservas_stock(fecha_reserva);
CREATE INDEX idx_movimientos_producto_fecha ON movimientos_stock(producto_id, fecha_movimiento);
CREATE INDEX idx_movimientos_documento ON movimientos_stock(documento_ref);
CREATE INDEX idx_movimientos_tipo_fecha ON movimientos_stock(tipo_movimiento, fecha_movimiento);
CREATE INDEX idx_precios_compra_producto_fecha_precio ON precios_compra(producto_id, fecha_precio,
precio_compra);
CREATE INDEX idx_precios_venta_producto_fecha_precio ON precios_venta(producto_id, fecha_precio, precio_venta);
CREATE INDEX idxCola_estado_fecha ON cola_compras_log(estado_procesamiento, fecha_evento);

```

5. Índices y justificación

Índice	Tabla	Justificación
productos(codigo) UNIQUE	PRODUCTOS	Evita productos duplicados y acelera consultas por código.
reservas_stock(producto_id, estado_reserva)	RESERVAS_STOCK	Permite buscar reservas pendientes por producto.
reservas_stock(documento_ref)	RESERVAS_STOCK	Permite encontrar reservas asociadas a una factura o pedido.
movimientos_stock(producto_id, fecha_movimiento)	MOVIMIENTOS_STOCK	Optimiza el historial/kardex por producto.
movimientos_stock(documento_ref)	MOVIMIENTOS_STOCK	Permite auditar movimientos por factura, compra o reserva.
precios_compra(producto_id, fecha_precio, precio_compra)	PRECIOS_COMPRA	Facilita consultar evolución de precios de compra.
precios_venta(producto_id, fecha_precio, precio_venta)	PRECIOS_VENTA	Facilita consultar evolución de precios de venta.
cola_compras_log(mensaje_id) UNIQUE	COLA_COMPRAS_LOG	Evita procesar dos veces el mismo mensaje.

6. Validaciones obligatorias

Validación	Dónde aplicar	Acción si falla
Código de producto obligatorio y único	API y BD	Rechazar registro.
Producto activo para reservar, vender o comprar	API	Rechazar operación.
Cantidad mayor a cero	API y BD	Rechazar operación.
Precio no negativo	API y BD	Rechazar operación.
Documento de referencia obligatorio	API y BD	Rechazar operación.
Stock suficiente para reservar	API + transacción BD	Responder error de stock insuficiente.
Reserva pendiente para confirmar o liberar	API + transacción BD	Rechazar si ya fue confirmada/liberada.
Mensaje de compra no procesado previamente	BD con mensaje_id UNIQUE	Ignorar duplicado o responder como ya procesado.

Validación	Dónde aplicar	Acción si falla
Identificadores positivos	Contrato OpenAPI	producto_id, reserva_id y demás IDs técnicos deben ser enteros mayores o iguales a 1.
Formato de documento	Contrato OpenAPI	documento_ref, factura_id y referencia_compra deben cumplir el formato PREFIJO-000-000000.
Cantidad positiva	Contrato OpenAPI	Las cantidades operativas de entrada deben ser decimal string positivo.
Cantidades de salida no negativas	Contrato OpenAPI	SalDOS, reservados, disponibles y stocks posteriores no deben admitir negativos.
Precios no negativos	Contrato OpenAPI	Los precios deben ser decimal string no negativo.

7. Funciones y endpoints

Nro.	Función	Entrada principal	Salida principal
1	Registrar producto	codigo, nombre, descripcion, categoria, unidad_medida	producto creado
2	Consultar producto	producto_id o codigo	datos del producto
3	Listar productos	filtros opcionales	lista de productos
4	Consultar stock	producto_id	cantidad_total, cantidad_reservada, cantidad_disponible
5	Crear reserva	documento_ref, producto_id, cantidad	reserva_id, estado, stock actualizado
6	Confirmar reserva	reserva_id o documento_ref	reserva confirmada, venta registrada

7	Liberar reserva	reserva_id, motivo	reserva liberada, disponible actualizado
8	Confirmar venta	factura con items	confirmación o error por stock
9	Procesar compra desde cola	mensaje_id, producto, cantidad, precio	estado de procesamiento
10	Consultar movimientos	producto_id, fechas opcionales	historial de movimientos

Endpoint	Uso	Ejemplo entrada	Ejemplo salida
POST /productos	Registrar producto	{ "codigo": "P001", "nombre": "Producto A", "categoria": "General", "unidad_medida": "UNIDAD" }	{ "producto_id": 1, "codigo": "P001", "estado": "CREADO" }
GET /productos/{producto_id}	Consultar producto	Path: producto_id=1	{ "producto_id": 1, "codigo": "P001", "nombre": "Producto A", "activo": true }
GET /productos?codigo=P001&activo=true	Listar productos	Query params opcionales	[{ "producto_id": 1, "codigo": "P001", "nombre": "Producto A" }]
GET /stock/{producto_id}	Consultar stock	Path: producto_id=1	{ "producto_id": 1, "cantidad_total": 10, "cantidad_reservada": 3, "cantidad_disponible": 7 }
POST /stock/reservas	Crear reserva	{ "documento_ref": "FV-001", "items": [{ "producto_id": 1, "cantidad": 2 }] }	{ "estado": "RESERVADO", "reservas": [{ "reserva_id": 10, "producto_id": 1, "cantidad_reservada": 2 }] }
POST /stock/reservas/{reserva_id}/confirmar	Confirmar reserva	Path: reserva_id=10	{ "estado": "CONFIRMADA", "movimiento_id": 50 }
POST /stock/reservas/{reserva_id}/liberar	Liberar reserva	{ "motivo_liberacion": "Cancelación de venta" }	{ "estado": "LIBERADA", "cantidad_liberada": 2 }
POST /ventas/confirmar	Confirmar venta completa	{ "factura_id": "FV-001", "items": [{ "producto_id": 1, "cantidad": 2, "precio_venta": 15000 }] }	{ "estado": "CONFIRMADA", "documento_ref": "FV-001" }

8. Ejemplos JSON de endpoints

Crear reserva

POST /stock/reservas

```
{
  "documento_ref": "FV-001-000123",
  "items": [
    { "producto_id": 1, "cantidad": "2.000"},
    { "producto_id": 5, "cantidad": "1.000"}
  ]
}
```

Respuesta OK

```
{
  "estado": "RESERVADO",
  "documento_ref": "FV-001-000123",
  "reservas": [
    { "reserva_id": 101, "producto_id": 1, "cantidad_reservada": 2, "cantidad_disponible": 8},
    { "reserva_id": 102, "producto_id": 5, "cantidad_reservada": 1, "cantidad_disponible": 4}
  ]
}
```

Respuesta con error

```
{
  "estado": "ERROR",
  "codigo": "STOCK_INSUFICIENTE",
  "mensaje": "No existe stock suficiente para el producto 5."
}
```

Confirmar reserva

```
POST /stock/reservas/101/confirmar
{
  "documento_ref": "FV-001-000123",
  "precio_venta": "15000.00"
}
```

Respuesta

```
{
  "estado": "CONFIRMADA",
  "reserva_id": 101,
  "producto_id": 1,
  "cantidad_descontada": 2,
  "stock_total_actual": 8,
  "cantidad_reservada_actual": 0,
  "cantidad_disponible_actual": 8
}
```

Procesar compra desde cola

```
POST /compras/eventos
Mensaje recibido desde cola RabbitMQ
{
  "mensaje_id": "CMP-2026-0001",
  "referencia_compra": "OC-001-000555",
  "producto_id": 1,
  "cantidad": 10,
  "precio_compra": "9000.00",
}
```

Respuesta lógica del procesamiento

```
{
  "mensaje_id": "CMP-2026-0001",
  "estado_procesamiento": "PROCESADO",
  "producto_id": 1,
  "stock_total_actual": 18
}
```

Validaciones reforzadas del contrato OpenAPI

El contrato OpenAPI debe declarar restricciones explícitas para evitar ambigüedades entre sistemas consumidores.

Reglas:

- Los identificadores técnicos como `producto\_id`, `reserva\_id` y `movimiento\_id` deben ser enteros positivos.
- Las referencias documentales como `documento\_ref`, `factura\_id` y `referencia\_compra` deben seguir el formato `PREFIXO-000-000000`.
- Las cantidades de entrada deben enviarse como string decimal positivo.
- Las cantidades de salida deben exponerse como string decimal no negativo.
- Los precios deben exponerse como string decimal no negativo.
- No se deben usar valores `number`, `double` ni `floating point` para cantidades o precios.

9. Procesos principales paso a paso

9.1 Crear reserva de stock

- Recibir documento de venta y lista de productos/cantidades.
- Validar que el documento de referencia no esté vacío.
- Validar que todos los productos existan y estén activos.
- Iniciar transacción.
- Bloquear las filas de STOCK_ACTUAL de los productos involucrados con SELECT ... FOR UPDATE.
- Validar que cantidad_disponible sea suficiente para cada ítem.
- Incrementar cantidad_reservada en STOCK_ACTUAL.
- Insertar registros en RESERVAS_STOCK con estado PENDIENTE.
- Insertar movimientos de tipo RESERVA si se desea auditar la reserva como movimiento.
- Confirmar transacción.
- Responder con las reservas creadas y las nuevas cantidades disponibles.

SQL lógico de creación de reserva

```
BEGIN;

SELECT producto_id, cantidad_total, cantidad_reservada, cantidad_disponible
FROM stock_actual
WHERE producto_id = 1
FOR UPDATE;

-- Validar en la aplicación: cantidad_disponible >= 2

UPDATE stock_actual
SET cantidad_reservada = cantidad_reservada + 2,
    fecha_ultima_actualizacion = CURRENT_TIMESTAMP,
    updated_by = 'sistema'
WHERE producto_id = 1;

INSERT INTO reservas_stock(producto_id, documento_ref, cantidad_reservada, estado_reserva, created_by)
VALUES (1, 'FV-001-000123', 2, 'PENDIENTE', 'ventas');

INSERT INTO movimientos_stock(producto_id, tipo_movimiento, origen, documento_ref, cantidad,
                                stock_anterior, stock_posterior, observacion, created_by)
VALUES (1, 'RESERVA', 'VENTAS', 'FV-001-000123', 2, 10, 10,
        'Reserva previa a confirmación de venta', 'ventas');

COMMIT;
```

9.2 Confirmar reserva y venta

- Recibir reserva_id o documento_ref.
- Iniciar transacción.
- Bloquear la reserva pendiente y el stock del producto.
- Validar que la reserva esté en estado PENDIENTE.
- Disminuir cantidad_total por la cantidad reservada.
- Disminuir cantidad_reservada por la misma cantidad.
- Cambiar estado de la reserva a CONFIRMADA.
- Registrar MOVIMIENTOS_STOCK de tipo VENTA.
- Registrar PRECIOS_VENTA.

- Confirmar transacción.

SQL lógico de confirmación de reserva

```
BEGIN;

SELECT *
FROM reservas_stock
WHERE reserva_id = 101 AND estado_reserva = 'PENDIENTE'
FOR UPDATE;

SELECT *
FROM stock_actual
WHERE producto_id = 1
FOR UPDATE;

UPDATE stock_actual
SET cantidad_total = cantidad_total - 2,
    cantidad_reservada = cantidad_reservada - 2,
    fecha_ultima_actualizacion = CURRENT_TIMESTAMP,
    updated_by = 'ventas'
WHERE producto_id = 1;

UPDATE reservas_stock
SET estado_reserva = 'CONFIRMADA',
    fecha_confirmacion = CURRENT_TIMESTAMP,
    updated_by = 'ventas'
WHERE reserva_id = 101;

INSERT INTO movimientos_stock(producto_id, reserva_id, tipo_movimiento, origen, documento_ref,
                             cantidad, stock_anterior, stock_posterior, observacion, created_by)
VALUES (1, 101, 'VENTA', 'VENTAS', 'FV-001-000123', 2, 10, 8,
        'Confirmación de venta desde reserva', 'ventas');
```

```
INSERT INTO precios_venta(producto_id, precio_venta, documento_ref)
VALUES (1, 15000, 'FV-001-000123');
```

```
COMMIT;
```

9.3 Liberar reserva

- Ocurre si la venta se cancela, falla, vence o cambia.
- Iniciar transacción.
- Bloquear la reserva pendiente y el stock del producto.
- Validar que la reserva esté PENDIENTE.
- Disminuir cantidad_reservada en STOCK_ACTUAL.
- Marcar la reserva como LIBERADA.
- Registrar movimiento LIBERACION_RESERVA.
- Confirmar transacción.

SQL lógico de liberación de reserva

```
BEGIN;

SELECT *
FROM reservas_stock
WHERE reserva_id = 101 AND estado_reserva = 'PENDIENTE'
FOR UPDATE;

SELECT *
FROM stock_actual
```

```

WHERE producto_id = 1
FOR UPDATE;

UPDATE stock_actual
SET cantidad_reservada = cantidad_reservada - 2,
    fecha_ultima_actualizacion = CURRENT_TIMESTAMP,
    updated_by = 'ventas'
WHERE producto_id = 1;

UPDATE reservas_stock
SET estado_reserva = 'LIBERADA',
    fecha_liberacion = CURRENT_TIMESTAMP,
    motivo_liberacion = 'Cancelación de venta',
    updated_by = 'ventas'
WHERE reserva_id = 101;

INSERT INTO movimientos_stock(producto_id, reserva_id, tipo_movimiento, origen, documento_ref,
                             cantidad, stock_anterior, stock_posterior, observacion, created_by)
VALUES (1, 101, 'LIBERACION_RESERVA', 'VENTAS', 'FV-001-000123', 2, 10, 10,
        'Liberación de reserva por cancelación', 'ventas');

COMMIT;

```

10. Flujo completo con reserva y concurrencia

El control de concurrencia recomendado para la primera implementación es pesimista: dentro de una transacción se bloquea la fila de STOCK_ACTUAL correspondiente al producto. Así, dos ventas simultáneas no pueden reservar o descontar el mismo stock al mismo tiempo.

- Venta A intenta reservar 6 unidades del Producto 1.
- Venta B intenta reservar 5 unidades del mismo producto al mismo tiempo.
- Stock total = 10; reservado = 0; disponible = 10.
- Venta A inicia transacción y bloquea la fila de stock.
- Venta B debe esperar hasta que Venta A confirme o revierta.
- Venta A reserva 6; disponible queda 4; confirma transacción.
- Venta B continúa, vuelve a leer stock disponible = 4.
- Venta B solicita 5, pero no alcanza; se rechaza con stock insuficiente.
- Resultado: no existe sobreventa.

Mermaid - Concurrencia en reserva de stock

```

sequenceDiagram
    actor V1 as Ventas A
    actor V2 as Ventas B
    participant API as Servicio Stock
    participant DB as PostgreSQL

    V1->>API: Solicitar reserva Producto 1, cantidad 6
    API->>DB: BEGIN
    API->>DB: SELECT stock_actual WHERE producto_id=1 FOR UPDATE
    DB-->>API: Bloqueo concedido, disponible=10

    V2->>API: Solicitar reserva Producto 1, cantidad 5
    API->>DB: BEGIN
    API->>DB: SELECT stock_actual WHERE producto_id=1 FOR UPDATE
    Note over V2,DB: Espera porque Venta A bloqueó la fila

    API->>DB: UPDATE cantidad_reservada = cantidad_reservada + 6
    API->>DB: INSERT RESERVAS_STOCK(PENDIENTE)

```

```

API->>DB: COMMIT
API-->>V1: Reserva creada

DB-->>API: Venta B continúa, disponible=4
API->>API: Validar 4 >= 5
API-->>V2: Error: stock insuficiente
API->>DB: ROLLBACK

```

11. Flujos escritos y Mermaid

11.1 Venta con reserva

- Ventas envía factura o pedido con productos y cantidades.
- Stock valida disponibilidad real considerando reservas previas.
- Stock crea reserva si hay disponibilidad.
- Ventas confirma la operación.
- Stock confirma la reserva, descuenta stock total y registra movimiento de venta.
- Si la operación se cancela, Stock libera la reserva.

Mermaid - Venta con reserva

```

sequenceDiagram
    actor Ventas
    participant API as Servicio Stock
    participant DB as Base de Datos

    Ventas->>API: Enviar factura/pedido con productos y cantidades
    API->>DB: Iniciar transacción
    API->>DB: Bloquear STOCK_ACTUAL por producto
    API->>DB: Consultar cantidad_disponible
    DB-->>API: Stock disponible
    alt Stock suficiente
        API->>DB: Crear RESERVAS_STOCK(PENDIENTE)
        API->>DB: Aumentar cantidad_reservada
        API->>DB: Confirmar transacción
        API-->>Ventas: Reserva creada
        Ventas->>API: Confirmar venta
        API->>DB: Iniciar transacción
        API->>DB: Bloquear reserva y stock
        API->>DB: Descontar cantidad_total y cantidad_reservada
        API->>DB: Marcar reserva CONFIRMADA
        API->>DB: Insertar MOVIMIENTOS_STOCK(VENTA)
        API->>DB: Insertar PRECIOS_VENTA
        API->>DB: Confirmar transacción
        API-->>Ventas: Venta confirmada
    else Stock insuficiente
        API->>DB: Revertir transacción
        API-->>Ventas: Error de stock
    end
end

```

11.2 Compra desde cola

- Compras publica un mensaje en la cola.
- El listener/consumer del módulo de stock lee el mensaje.
- Se valida que mensaje_id no haya sido procesado.
- Se inicia transacción, se actualiza stock y se registran movimiento y precio.
- Se marca el mensaje como PROCESADO y se confirma el consumo.

Mermaid - Compra desde cola

```
sequenceDiagram
    actor Compras
    participant Cola as RabbitMQ
    participant API as Consumer Stock
    participant DB as Base de Datos

    Compras->>Cola: Publicar mensaje de compra
    Cola-->>API: Entregar mensaje
    API->>DB: Verificar mensaje_id en COLA_COMPRAS_LOG
    alt Mensaje no procesado
        API->>DB: Iniciar transacción
        API->>DB: Bloquear STOCK_ACTUAL del producto
        API->>DB: Incrementar cantidad_total
        API->>DB: Insertar MOVIMIENTOS_STOCK(COMPRAS)
        API->>DB: Insertar PRECIOS_COMPRA
        API->>DB: Registrar COLA_COMPRAS_LOG(PROCESADO)
        API->>DB: Confirmar transacción
        API-->>Cola: Confirmar mensaje
    else Mensaje duplicado
        API-->>Cola: Confirmar sin reprocesar
    end
end
```

12. Logs y auditoría simple

La primera implementación no incluye seguridad completa, pero sí debe registrar información mínima de auditoría para rastrear operaciones. Esta auditoría puede usar campos simples como `created_by`, `updated_by`, `created_at` y `updated_at`. Si todavía no existe autenticación, el valor puede ser el sistema origen: VENTAS, COMPRAS, STOCK o SISTEMA.

Evento	Nivel log	Datos mínimos
Producto creado	INFO	codigo, producto_id, origen
Reserva creada	INFO	documento_ref, reserva_id, producto_id, cantidad
Venta confirmada	INFO	documento_ref, producto_id, cantidad, movimiento_id
Reserva liberada	INFO	documento_ref, reserva_id, motivo
Compra procesada	INFO	mensaje_id, referencia_compra, producto_id, cantidad
Stock insuficiente	WARNING	documento_ref, producto_id, cantidad solicitada, disponible
Error de base de datos	ERROR	operación, detalle técnico interno
Mensaje duplicado	WARNING	mensaje_id, referencia_compra

13. Criterios de aceptación

- No se debe confirmar una venta sin stock suficiente.
- No se debe reservar más de la cantidad disponible.
- Una reserva confirmada debe descontar el stock total y reservado.
- Una reserva liberada debe disminuir la cantidad reservada y aumentar la disponible.
- Toda compra válida debe incrementar el stock y registrar movimiento de compra.
- Un mensaje de compra duplicado no debe incrementar dos veces el stock.
- Toda operación que afecte stock debe tener documento de referencia.
- Toda operación que afecte stock debe dejar movimiento registrado.
- Si ocurre error dentro de una operación, la transacción debe revertirse.

14. Orden recomendado de desarrollo

Orden	Actividad	Resultado
1	Crear proyecto FastAPI y conexión PostgreSQL	Base técnica funcionando.
2	Crear tablas y migraciones Alembic	Modelo persistente listo.
3	Implementar productos	Alta, consulta y listado.
4	Implementar stock actual	Consulta de saldos.
5	Implementar compra desde cola simulada	Entrada de stock y movimientos.
6	Implementar reservas	Reserva, confirmación y liberación.
7	Implementar confirmación de venta	Salida definitiva y precio de venta.
8	Implementar logs y manejo de errores	Diagnóstico operativo.
9	Agregar pruebas críticas	Validación de reglas principales.

15. Contrato OpenAPI

El contrato de servicios se genera automáticamente desde el proyecto mediante el script correspondiente. El archivo `openapi.yaml` no debe editarse manualmente como fuente principal, ya que puede ser sobrescrito al regenerar la especificación.

La documentación OpenAPI debe incluir metadata operativa para facilitar la integración entre módulos:

- título de la API;
- descripción general;
- versión;
- servidores disponibles;
- contacto responsable;
- licencia o uso declarado;
- términos de servicio;
- documentación externa;
- formato estándar de errores;
- enums explícitos para campos controlados.

El contrato no debe incluir campos de ejemplos JSON. Los precios se exponen como valores numéricos simples y la interpretación monetaria queda fuera del alcance del módulo de stock..

16. Enums explícitos del contrato

Los campos que representan estados, tipos, orígenes y códigos de error no deben documentarse como textos libres. Deben definirse como enums dentro del contrato OpenAPI.

Esto aplica a:

- estado;
- código de error;
- tipo de movimiento;
- origen del movimiento;
- estado de procesamiento;
- unidad de medida.

No se define enum de moneda porque el contrato no manejará moneda.

17. Modelo decimal del contrato de servicios

El contrato OpenAPI no debe exponer cantidades ni precios como ``number``, ``double`` o ``floating point``.

Las cantidades y precios deben documentarse como ``string`` con formato decimal para evitar errores de precisión en los sistemas consumidores.

Reglas del contrato:

- Las cantidades de stock se representan como string decimal con hasta 3 decimales.
- Los precios se representan como string decimal con hasta 2 decimales.
- El contrato no maneja moneda.
- Los precios son valores decimales simples.
- La interpretación monetaria queda fuera del alcance del módulo de stock.
- La base de datos puede usar tipos decimales exactos, pero el contrato de servicios no debe usar ``double``.

Ejemplo de cantidad:

```
```json
{
 "producto_id": 1,
 "cantidad": "10.000"
}
```

## 18. OperationId estables

El contrato OpenAPI debe definir ``operationId`` estables, cortos y legibles para cada operación.

No se deben usar los nombres internos generados automáticamente por el framework, porque esos nombres pueden ser extensos, poco claros y dependientes de la estructura interna del código.

Los ``operationId`` deben cumplir estas reglas:

- usar nombres en estilo camelCase;
- representar claramente la acción de negocio;
- mantenerse estables entre versiones;
- no depender del path interno ni del nombre de la función Python;
- facilitar la generación automática de SDKs y clientes.

Ejemplos:

- ``crearProducto``
- ``listarProductos``
- ``obtenerProducto``
- ``obtenerStock``
- ``crearReserva``

- `confirmarReserva`
- `liberarReserva`
- `procesarEventoCompra`
- `confirmarVenta`

## 19. Clasificación de operaciones del contrato

El contrato OpenAPI puede incluir operaciones de uso general y operaciones de uso interno.

Las operaciones internas forman parte del sistema y se documentan para mantener completo el contrato funcional, pero no deben considerarse servicios públicos para integradores externos.

Se consideran operaciones internas aquellas que:

- modifican estados intermedios del proceso de stock;
- son invocadas exclusivamente por módulos internos del ecosistema SDI;
- no deben ser utilizadas por consumidores externos ni por clientes ajenos al sistema.

Para la primera implementación, se clasifican como internas:

- POST /stock/reservas/{reserva\_id}/confirmar
- POST /stock/reservas/{reserva\_id}/liberar

Estas operaciones deben figurar en el contrato con una marca explícita de uso interno.

Su publicación en OpenAPI tiene fines documentales y de integración entre módulos internos, no de exposición pública.



## 20. Política normalizada de errores

Todas las respuestas de error del contrato OpenAPI deben usar el mismo schema:

```
{
 "estado": "ERROR",
 "codigo": "VALIDACION_ERROR",
 "mensaje": "campo: descripción clara de la validación incumplida."
}
```

HTTP	USO
400	Error funcional o regla de negocio inválida.
401	Solicitud no autenticada o credencial inválida.
403	Consumidor autenticado sin permiso para ejecutar la operación.
404	Recurso solicitado inexistente.
409	Conflicto de estado o concurrencia funcional.
422	Error de validación del contrato OpenAPI.
500	Error interno controlado del servicio.
DEFAULT	Error no clasificado por el contrato.

## 21. Idempotencia, replay y concurrencia en el contrato

El contrato OpenAPI debe expresar reglas claras para reintentos, duplicados y operaciones concurrentes.

Las operaciones que modifican stock, reservas o ventas deben ser determinísticas ante reintentos. Esto evita que un consumidor duplique compras, reservas, movimientos o descuentos de stock por reenviar una solicitud.

Reglas:

- `POST /compras/eventos` usa `mensaje\_id` como clave idempotente.
- `POST /stock/reservas` usa `documento\_ref` como clave idempotente.
- `POST /ventas/confirmar` usa `factura\_id` como clave idempotente.
- `POST /stock/reservas/{reserva\_id}/confirmar` usa `reserva\_id` como clave de operación.
- `POST /stock/reservas/{reserva\_id}/liberar` usa `reserva\_id` como clave de operación.

Si se recibe la misma clave con el mismo contenido, la operación no debe duplicar efectos.

Si se recibe la misma clave con contenido diferente, debe responderse con error `409` y código `REPLAY\_NO\_COINCIDE`.

Las operaciones que afectan stock deben declarar una política de concurrencia. Ante conflictos de modificación simultánea, el contrato debe responder `409` con código `CONFLICTO\_CONCURRENCIA`.